

Guía de Despliegue - Numbrica Backend

Preparación para Despliegue

Prerrequisitos

- Node.js 18+
- PostgreSQL 14+
- Python 3.9+ con dependencias de motores instaladas
- Acceso a servidor de producción

Despliegue en Producción

1. Variables de Entorno

Crear archivo `.env` en producción con:

```
# Database (PostgreSQL en producción)
DATABASE_URL=postgresql://user:password@host:5432/numbrica_prod

# Application
PORT=3000
NODE_ENV=production

# Python Engines (rutas absolutas en servidor)
PYTHON_ENGINES_PATH=/opt/numbrica/engines
PYTHON_TEMPLATES_PATH=/opt/numbrica/templates

# CORS (dominio del frontend)
CORS_ORIGIN=https://numbrica.com
```

2. Instalación de Dependencias

```
# Clonar o copiar el proyecto
cd /opt/numbrica_backend

# Instalar dependencias
yarn install --production

# Generar Prisma Client
yarn prisma generate

# Sincronizar base de datos
yarn prisma db push
```

3. Compilar Aplicación

```
yarn build
```

Esto genera la carpeta `dist/` con el código compilado.

4. Iniciar en Producción

Opción A: Directamente con Node

```
node dist/main.js
```

Opción B: Con PM2 (recomendado)

```
# Instalar PM2 globalmente
npm install -g pm2

# Iniciar aplicación
pm2 start dist/main.js --name numbrica-api

# Ver logs
pm2 logs numbrica-api

# Monitoreo
pm2 monit

# Autostart en reboot
pm2 startup
pm2 save
```

Opción C: Con systemd

Crear archivo `/etc/systemd/system/numbrica-api.service` :

```
[Unit]
Description=Numbrica API
After=network.target postgresql.service

[Service]
Type=simple
User=numbrica
WorkingDirectory=/opt/numbrica_backend
Environment=NODE_ENV=production
ExecStart=/usr/bin/node /opt/numbrica_backend/dist/main.js
Restart=on-failure
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Luego:

```
sudo systemctl daemon-reload
sudo systemctl enable numbrica-api
sudo systemctl start numbrica-api
sudo systemctl status numbrica-api
```

Configuración de Nginx (Reverse Proxy)

Crear archivo `/etc/nginx/sites-available/numbrica-api`:

```
server {
    listen 80;
    server_name api.numbrica.com;

    # Redirigir a HTTPS
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name api.numbrica.com;

    # Certificados SSL (Let's Encrypt)
    ssl_certificate /etc/letsencrypt/live/api.numbrica.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.numbrica.com/privkey.pem;

    # Configuración SSL
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    # Logs
    access_log /var/log/nginx/numbrica-api-access.log;
    error_log /var/log/nginx/numbrica-api-error.log;

    # Proxy a la aplicación NestJS
    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }

    # Límites de tamaño para requests
    client_max_body_size 10M;
}
```

Activar configuración:

```
sudo ln -s /etc/nginx/sites-available/numbrica-api /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

Base de Datos PostgreSQL

Backup

```
# Backup completo
pg_dump -U numbrica_user -d numbrica_prod -F c -f backup_$(date +%Y%m%d).dump

# Backup solo schema
pg_dump -U numbrica_user -d numbrica_prod --schema-only > schema.sql
```

Restore

```
pg_restore -U numbrica_user -d numbrica_prod -v backup_20250118.dump
```

Monitoreo

```
-- Verificar conexiones activas
SELECT * FROM pg_stat_activity WHERE datname = 'numbrica_prod';

-- Tamaño de la base de datos
SELECT pg_size_pretty(pg_database_size('numbrica_prod'));

-- Tablas más grandes
SELECT
    schemaname,
    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS size
FROM pg_tables
WHERE schemaname = 'public'
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC;
```



Instalación de Dependencias Python

En el servidor de producción:

```
# Instalar Python y pip
sudo apt update
sudo apt install python3 python3-pip

# Instalar dependencias de los motores
sudo pip3 install pyswisseph

# Verificar instalación
python3 -c "import swisseph as swe; print('✅ pyswisseph instalado:', swe.version)"
```

Copiar motores y plantillas:

```
# Crear directorios
sudo mkdir -p /opt/numbrica/engines
sudo mkdir -p /opt/numbrica/templates

# Copiar archivos
sudo cp /path/to/numbrica_engines/*.py /opt/numbrica/engines/
sudo cp /path/to/numbrica_templates/*.json /opt/numbrica/templates/

# Permisos
sudo chown -R numbrica:numbrica /opt/numbrica
```



Monitoreo y Logs

Logs de Aplicación

```
# Con PM2
pm2 logs numbrica-api

# Con systemd
sudo journalctl -u numbrica-api -f

# Archivo directo (si se configura)
tail -f /var/log/numbrica/api.log
```

Métricas

Endpoints de monitoreo:

- **Health Check:** GET /api/health
- **Documentación:** GET /api-docs

Integrar con herramientas de monitoreo:

- Prometheus
- Grafana
- New Relic
- DataDog



Seguridad

Checklist de Seguridad

- [] Variables de entorno seguras (no en código)
- [] Base de datos con usuario de permisos limitados
- [] SSL/TLS configurado (HTTPS)
- [] CORS configurado correctamente
- [] Rate limiting en Nginx
- [] Firewall configurado (UFW/iptables)
- [] Actualizaciones de seguridad automáticas
- [] Logs monitoreados
- [] Backups automáticos configurados

Rate Limiting en Nginx

Agregar en configuración de Nginx:

```
# Definir zona de rate limiting
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;

server {
    # ... configuración SSL ...

    location /api/calculate {
        limit_req zone=api_limit burst=5 nodelay;
        proxy_pass http://localhost:3000;
        # ... otros proxy_set_header ...
    }
}
```



Actualización de Código

Proceso de actualización sin downtime:

```
# 1. Pull nuevo código
cd /opt/numbrica_backend
git pull origin main

# 2. Instalar nuevas dependencias
yarn install

# 3. Compilar
yarn build

# 4. Migrar base de datos (si hay cambios)
yarn prisma db push

# 5. Reload aplicación
pm2 reload numbrica-api

# 6. Verificar
curl http://localhost:3000/api/health
```



Escalabilidad

Configuración Multi-Instancia con PM2

```
# Iniciar múltiples instancias (cluster mode)
pm2 start dist/main.js -i max --name numbrica-api

# 0 especificar número de instancias
pm2 start dist/main.js -i 4 --name numbrica-api
```

Load Balancing con Nginx

```
upstream numbrica_backend {  
    least_conn;  
    server localhost:3000;  
    server localhost:3001;  
    server localhost:3002;  
    server localhost:3003;  
}  
  
server {  
    # ... configuración SSL ...  
  
    location / {  
        proxy_pass http://numbrica_backend;  
        # ... proxy_set_header ...  
    }  
}
```

Troubleshooting

Aplicación no inicia

```
# Verificar logs  
pm2 logs numbrica-api --lines 100  
  
# Verificar puerto  
sudo netstat -tulpn | grep :3000  
  
# Verificar variables de entorno  
pm2 env numbrica-api
```

Base de datos no conecta

```
# Verificar PostgreSQL  
sudo systemctl status postgresql  
  
# Verificar credenciales  
psql -U numbrica_user -d numbrica_prod -h localhost  
  
# Verificar en logs de aplicación  
grep "PrismaService" /var/log/numbrica/api.log
```

Python engines fallan

```
# Verificar Python
python3 --version

# Verificar pyswisseph
python3 -c "import swisseph"

# Verificar rutas en .env
echo $PYTHON_ENGINES_PATH
ls -la $PYTHON_ENGINES_PATH
```

✓ Checklist de Despliegue

- ☐ Variables de entorno configuradas
- ☐ Base de datos creada y migrada
- ☐ Dependencias Node instaladas
- ☐ Dependencias Python instaladas
- ☐ Motores y plantillas copiados
- ☐ Aplicación compilada
- ☐ PM2/systemd configurado
- ☐ Nginx configurado con SSL
- ☐ DNS apuntando al servidor
- ☐ Backups configurados
- ☐ Monitoreo configurado
- ☐ Health check respondiendo
- ☐ Swagger accesible
- ☐ Tests end-to-end pasando

¡Numbrica Backend listo para producción! 🚀🌐